

Designing an Analytics Database to Integrate Research Experiments and Laboratory Assets

Background Information:

In the biotechnology industry, working with laboratory assets and experimental data is very common. Laboratory assets are used to support scientific research, quality activities, and product development. These assets are typically managed through an enterprise asset management (EAM) system, which tracks current and historical information such as calibration records, maintenance activities, validation status, asset location, and equipment movement (when instruments are shipped to other sites for use). From experience, these laboratory assets play a critical role in research experiments and directly impact the quality and reliability of experimental results. Experimental data, on the other hand, is usually managed in electronic lab notebook (ELN) or laboratory information management system (LIMS) platforms. These systems store information related to experiments, including experimental procedures, start and end dates, sample types, and supporting metadata. While both EAM systems and ELN/LIMS platforms are essential to laboratory operations, they are typically implemented as separate systems and are not designed to work together.

Based on experience in biotech environments, this separation often makes data gathering inefficient and time consuming, especially when information needs to be validated or reconciled across systems. In practice, analysts and laboratory teams frequently have to manually pull data from both systems in order to answer questions

that span asset usage and experimental outcomes. This process is not only slow, but it also increases the likelihood of errors.

To make analysis more efficient, this project proposes the design of an Analytics Database that integrates data from the EAM system and the ELN/LIMS system. By bringing these datasets together into a single analytics database, it becomes easier to retrieve information and answer more complex, cross domain questions. For example, the system could help identify whether certain laboratory assets are consistently associated with experiments that fail or experience delays, something that is difficult to analyze when data is being gathered from different domains.

The database will be managed using a relational DBMS (MySQL) and accessed through an application written in Python support analytics and visualizations. These results can be connected to business intelligence tools such as Power BI or Tableau for dashboarding. The goal of this project is to demonstrate how the Analytics Database can improve traceability, operational visibility, efficiency, and less error prone due to manual cross references.

Business Problems:

EAM and ELN/LIMS systems are not unified:

Laboratory asset data and experimental data are stored in separate operational systems that do not share a common database. As a result, data from EAM and ELN/LIMS systems is not easily accessible in a unified way. Answering questions that require information from both domains often involves manually pulling data from multiple sources and combining it, which is inefficient and difficult to maintain.

Limited organization/visibility of asset usage across experiments and departments:

There is limited visibility into how laboratory assets are used across different experiments and departments such as research, development, engineering, and pharmacology. Without a structured way to link assets to experiments, it is difficult to analyze usage patterns, understand equipment sharing across teams, or measure how assets support different types of work.

Challenges validating and analyzing cross-domain data:

Even after data is retrieved from multiple systems, validating and analyzing it remains challenging. Cross checking information such as maintenance status, calibration due dates, asset criticality, and experiment timelines is difficult when data originates from separate sources. This makes it harder to identify whether overdue or high-risk assets were used in critical experiments and increases the risk of errors during analysis.

Difficulty supporting audits, compliance, and traceability requirements:

The lack of integration between asset and experiment data makes it difficult to support audits, compliance reviews, and validation activities. Without clear traceability linking assets, experiments, and dates, organizations face increased effort, disorganization, and potential inconsistencies when preparing documentation for regulatory or quality related audits.

Business Requirements:

Data Integration Requirements:

- The analytics database system should bring together laboratory asset data from the EAM system and experimental data from the ELN/LIMS system into a single place for analysis.
- The system should allow both current and historical data from EAM and ELN/LIMS to be queried together so information from both systems can be analyzed at the same time.

EAM Requirements:

- The analytics database system should store a unique identifier for each laboratory asset.
- Asset information should include asset type, current status, location, department (individual department or shared), and overall maintenance status.
- Maintenance information should be available for each asset, including maintenance type (calibration, preventive maintenance, validation, cleaning, installation, or relocation), maintenance dates, and outcomes.
- The system should make it possible to identify assets that are overdue for maintenance or calibration and link that information to the experiments in which the assets were used in.

ELN/LIMS Requirements:

- The analytics database system should store experiment information such as experimentID, experiment type (research, development, pharmacology, protein

engineering), department, start and end dates, experiment status, and the laboratory assets used.

- Sample information should also be included, such as sampleID and sample type, and should be linked to the associated experiment.
- The system should allow users to see experiments across different departments and the equipment used for those experiments.

Laboratory Asset and Experiment Traceability Requirements:

- The analytics database system should support linking an experiment to one or more laboratory assets, and a single laboratory asset should be able to be associated with multiple experiments.
- It should be possible to analyze which assets were used for specific experiments and how laboratory assets are used across departments.
- The system should support identifying experiments that used assets with overdue maintenance or calibration.

Compliance, Validation, Audit, and Traceability Requirements:

- The analytics database system should support cross domain analysis that combines asset status, maintenance history (calibration, installation, validation, preventive maintenance, and relocations), and experiment timelines.
- This should make it easier to confirm whether assets were in an acceptable state at the time experiments were performed and support audits and compliance reviews.
- The system should allow reports to be generated that clearly link asset records with experiment records for regulatory or quality related reviews.

Reporting and Application Requirements:

- The analytics database system should be implemented using a relational DBMS such as MySQL.
- A Python application should be able to connect to the database system and run SQL queries and support analytics and visualizations.
- Data should be retrievable in a format that can be used for further analysis or dashboarding if needed (through PowerBI/Tableau).

Charlotte Sy
DADS6700
Milestone#1

Introduction:

Modern biotechnology laboratories rely on multiple operational systems to manage critical information. Enterprise Asset Management (EAM) systems track laboratory equipment, including maintenance history, calibration records, validation status, location, and ownership. At the same time, Electronic Lab Notebook (ELN) and Laboratory Information Management Systems (LIMS) store experimental data such as experiments, timelines, departments, and sample information. Although both systems are essential to laboratory operations, they are typically implemented independently making cross-domain analysis difficult and inefficient.

This project proposes the design of an analytics database that integrates asset and experiment data into a unified relational database. The goal is to improve traceability, improve cross functional analysis, and support audit and compliance requirements. In Milestone 1 the conceptual model (ER/EER/UML) is designed then mapping to a normalized relational model. The design supports asset and experiment data traceability which can be used for a variety of data analysis and reporting.

Business Rules

ASSET:

- Each laboratory asset has a unique asset_id.
- Each asset has asset_type, asset_current_status, and asset_maintenance_status.
- Each asset is associated with exactly one department and one location.
- Assets include both current and historical records.
- A department can be associated with zero or many assets.
- A location can be associated with zero or many assets.

MAINTENANCE:

- Each maintenance record has a unique maintenance_id.
- Each maintenance record belongs to exactly one asset.
- Each asset can have zero or many maintenance records.
- Each maintenance record includes maintenance_date and maintenance_result.
- Each maintenance record is associated with exactly one maintenance type.
- A maintenance type can be associated with zero or many maintenance records.

EXPERIMENT:

- Each experiment has a unique experiment_id.
- Each experiment includes experiment_start_date, experiment_end_date, and experiment_status.
- Each experiment is associated with exactly one experiment type and one department.
- An experiment type can be associated with zero or many experiments.
- A department can be associated with zero or many experiments.

SAMPLE:

- Each sample has a unique sample_id.
- Each sample is linked to exactly one experiment.
- Each experiment can have zero or many samples.
- Each sample is associated with exactly one sample type.
- A sample type can be associated with zero or many samples.

ASSET_EXPERIMENT:

- Asset_Experiment is the associative entity between Experiment and Asset.
- Each record in Asset_Experiment links one experiment to one asset.
- Each experiment must be associated with one or more assets.
- Each asset can be associated with zero or many experiments.
- Each record includes usage_start_date and usage_end_date.

REFERENCE TABLES:

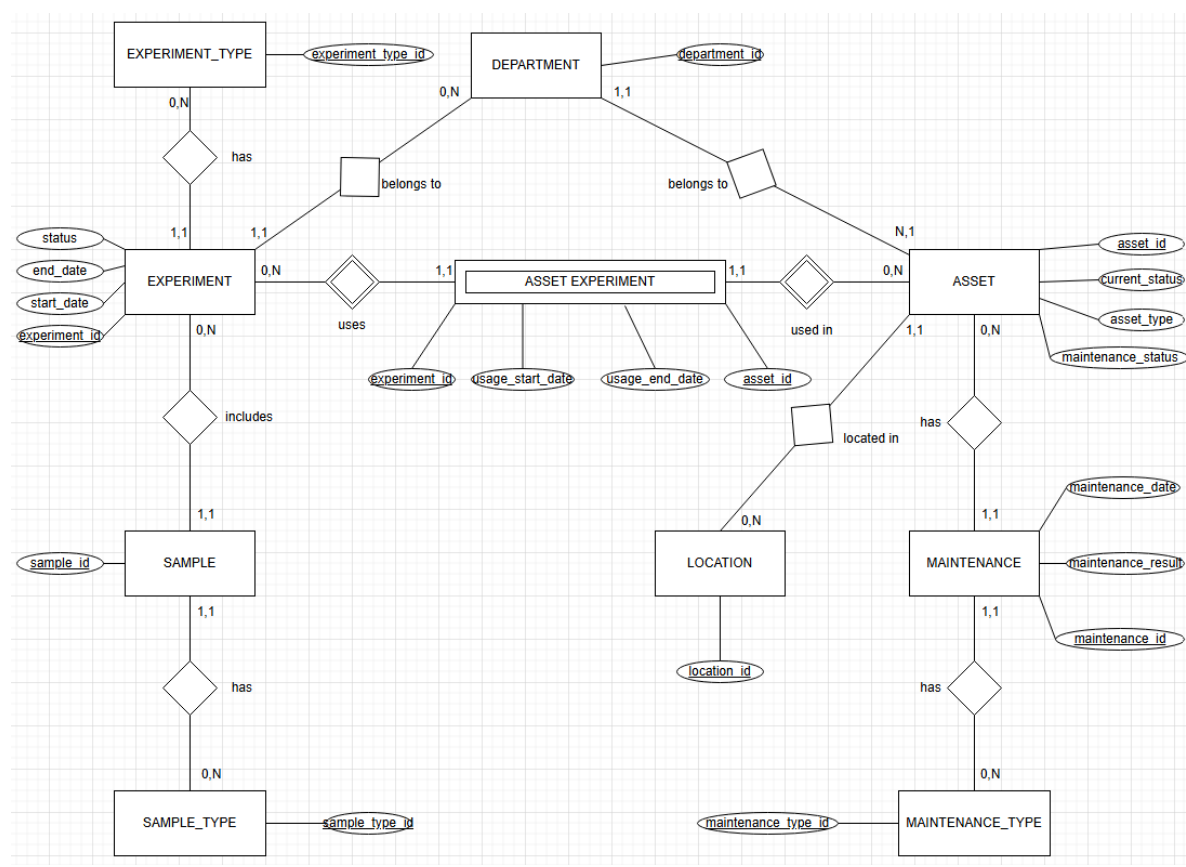
- Each department has a unique department_id.
- Each location has a unique location_id.
- Each experiment type has a unique experiment_type_id.
- Each sample type has a unique sample_type_id.
- Each maintenance type has a unique maintenance_type_id.
- These tables store values and are referenced by the main entities to ensure consistency and reduce redundancy.

Cardinality/Participation RULES:

- Each experiment can use one or more laboratory assets (1:N).

- Each laboratory asset can be used in multiple experiments (0:N).
- Associative entity Asset Experiment for Experiment and Asset.
- The system should be able to identify which experiments used assets with pass due maintenance (maintenance date and usage start date).
- The system supports analysis of assets used for different departments.

Conceptual Design (EER Model)



The conceptual model above was developed using an Enhanced Entity Relationship (EER) to capture the structure and relationship requirements of the integrated database (EAM/ELN/LIMS). The model reflects the main entities Experiment, Asset, Sample,

Maintenance, and Asset_Experiment, along with the reference entities Department, Location, Experiment_Type, Sample_Type, and Maintenance_Type.

EXPERIMENT/SAMPLE:

The Experiment entity represents laboratory studies and includes attributes experiment_id, experiment_start_date, experiment_end_date, and experiment_status. Each experiment is associated with exactly one Experiment_Type and one Department. An experiment may include zero or more samples (0,N), while each sample must belong to exactly one experiment (1,1).

ASSET:

The Asset entity represents laboratory equipment used during experiments. It includes attributes such as asset_id, asset_type, asset_current_status, and asset_maintenance_status. Each asset is associated with exactly one Department and one Location. Assets can exist independently of experiments and can be used across multiple experiments.

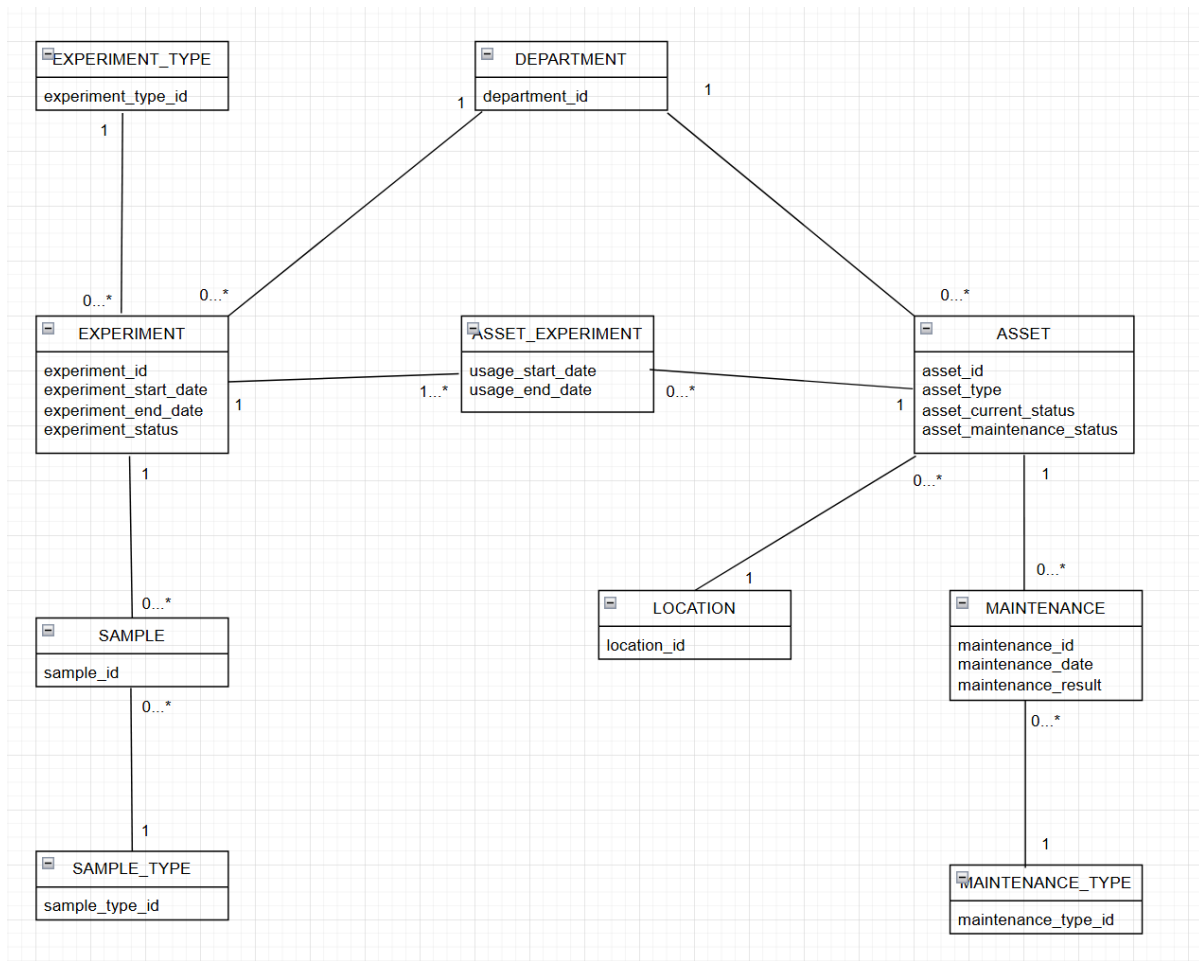
ASSET_EXPERIMENT:

The relationship between Experiment and Asset is M,N. This relationship is implemented using the associative entity Asset_Experiment in the relational model. Each experiment must use at least one asset (1,N), and an asset may be used in zero or many experiments (0,N). The Associative Entity Asset_Experiment is to capture the relationship between Experiment and Asset and includes attributes usage_start_date and usage_end_date. The attributes describe which asset was used in a specific experiment and the date of usage.

MAINTENANCE:

The Maintenance entity represents maintenance activities performed on assets. Each maintenance record must be associated with exactly one asset (1,1), and an asset can have zero or many maintenance records (0,N). Each maintenance record is also associated with exactly one Maintenance_Type.

UML Class Diagram:



The UML class diagram mirrors the conceptual EER model while presenting the system from an object oriented perspective. Each entity in the EER model is represented as a

class in the UML diagram: Experiment, Asset, Maintenance, Sample, Asset_Experiment, Department, Location, Experiment_Type, Sample_Type, and Maintenance_Type.

Associations between classes reflect the same multiplicities as defined in the EER model. The UML diagram confirms structural consistency between the conceptual and logical representations and ensures that multiplicities and relationships are preserved prior to relational mapping.

Relational Schema

DEPARTMENT(department_id PK)

LOCATION(location_id PK)

EXPERIMENT_TYPE(experiment_type_id PK)

SAMPLE_TYPE(sample_type_id PK)

MAINTENANCE_TYPE(maintenance_type_id PK)

EXPERIMENT(

experiment_id PK,

experiment_type_id FK REFERENCES EXPERIMENT_TYPE(experiment_type_id)

NOT NULL,

department_id FK REFERENCES DEPARTMENT(department_id) NOT NULL,

experiment_start_date NOT NULL,

experiment_end_date NULL,

experiment_status NOT NULL)

ASSET(

asset_id PK,

asset_type NOT NULL,

asset_current_status NOT NULL,

asset_maintenance_status NOT NULL,

department_id FK REFERENCES DEPARTMENT(department_id) NOT NULL,

location_id FK REFERENCES LOCATION(location_id) NOT NULL)

MAINTENANCE(

maintenance_id PK,

maintenance_type_id FK REFERENCES

MAINTENANCE_TYPE(maintenance_type_id) NOT NULL,

maintenance_date NOT NULL,

maintenance_result NOT NULL,

asset_id FK REFERENCES ASSET(asset_id) NOT NULL)

SAMPLE(

sample_id PK,

sample_type_id FK REFERENCES SAMPLE_TYPE(sample_type_id) NOT NULL,

experiment_id FK REFERENCES EXPERIMENT(experiment_id) NOT NULL)

ASSET_EXPERIMENT(

experiment_id PK FK REFERENCES EXPERIMENT(experiment_id) NOT NULL,

asset_id PK FK REFERENCES ASSET(asset_id) NOT NULL,

usage_start_date NOT NULL,

usage_end_date NOT NULL)

- **The relations satisfy 1NF, 2NF, and 3NF because attributes are atomic, no partial dependencies exist, and no transitive dependencies are present.**

- **Use of separate reference tables for department, location, experiment type, sample type, and maintenance type reduces redundancy and improves consistency across the database.**

Unsupported Cardinalities and Constraints

While the relational schema enforces structural relationships through primary and foreign keys, certain business constraints cannot be enforced.

- requirement that each experiment must use at least one asset cannot be strictly enforced through foreign key constraints alone.
- Temporal constraint on checking if the asset was compliant when it was used in an experiment.

Conclusion

Milestone one presents the development of a conceptual and logical data model for an analytics database integrating laboratory asset and experiment data. The EER model established entities, attributes, cardinalities, and participation constraints aligned with business requirements. The UML class diagram validated structural consistency from an object-oriented perspective. The relational schema was systematically derived and normalized to Third Normal Form to ensure data integrity and minimize redundancy through Primary keys, foreign keys, not nulls.

The resulting design supports asset-experiment traceability, maintenance validation analysis, and cross-departmental usage reporting. Although certain temporal and compliance constraints require application-level enforcement, the relational structure

provides a foundation for analytics and audit support for a biotech/pharmaceutical company.

Charlotte SY

DADS6700

Milestone#2

Step 1| Create a database in MySQL for your project.

- `CREATE DATABASE milestone2db;`
- `USE milestone2db;`

-- Reference Tables

- `CREATE TABLE department(
 department_id INT PRIMARY KEY,
 department_name VARCHAR(50));`
- `CREATE TABLE location(
 location_id INT PRIMARY KEY,
 location_name VARCHAR(50));`
- `CREATE TABLE experiment_type(
 experiment_type_id INT PRIMARY KEY,
 experiment_type_name VARCHAR(50));`
- `CREATE TABLE sample_type(
 sample_type_id INT PRIMARY KEY,
 sample_type_name VARCHAR(50));`
- `CREATE TABLE maintenance_type(
 maintenance_type_id INT PRIMARY KEY,
 maintenance_type_name VARCHAR(50));`

-- Strong Entity Tables

- `CREATE TABLE experiment(
 experiment_id INT PRIMARY KEY,
 experiment_type_id INT,
 department_id INT,
 start_date DATE,
 end_date DATE,
 status VARCHAR(20),
 FOREIGN KEY (experiment_type_id) REFERENCES experiment_type(experiment_type_id),
 FOREIGN KEY (department_id) REFERENCES department(department_id);`
- `CREATE TABLE asset(
 asset_id INT PRIMARY KEY,
 asset_type VARCHAR(50),
 asset_status VARCHAR(50),
 maintenance_status VARCHAR(50),
 department_id INT,
 location_id INT,
 FOREIGN KEY (department_id) REFERENCES department(department_id),
 FOREIGN KEY (location_id) REFERENCES location(location_id);`
- `CREATE TABLE sample(
 sample_id INT PRIMARY KEY,
 experiment_id INT,
 sample_type_id INT,
 collect_sample_date DATE,
 FOREIGN KEY (experiment_id) REFERENCES experiment(experiment_id),
 FOREIGN KEY (sample_type_id) REFERENCES sample_type(sample_type_id);`
- `CREATE TABLE maintenance(
 maintenance_id INT PRIMARY KEY,
 asset_id INT,
 maintenance_type_id INT,
 maintenance_date DATE,
 result VARCHAR(50),
 FOREIGN KEY (asset_id) REFERENCES asset(asset_id),
 FOREIGN KEY (maintenance_type_id) REFERENCES maintenance_type(maintenance_type_id);`

STEP 2 | Populate your tables with sample data.

```
-- Associative Table
CREATE TABLE asset_experiment(
    experiment_id INT,
    asset_id INT,
    usage_start_date DATE,
    usage_end_date DATE,
    PRIMARY KEY (experiment_id, asset_id),
    FOREIGN KEY (experiment_id) REFERENCES experiment(experiment_id),
    FOREIGN KEY (asset_id) REFERENCES asset(asset_id));
```

INSERT INTO department VALUES

(1, 'Biology'),

(2, 'Chemistry'),

(3, 'Pharmacology'),

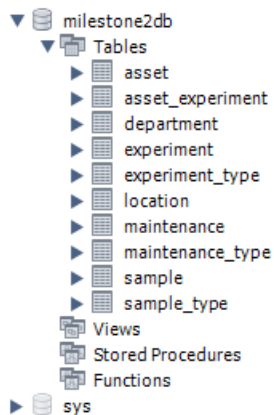
(4, 'R&D'),

(5, 'QA'),

(6, 'Manufacturing'),

(7, 'Clinical'),

(8, 'Microbiology'),



(9, 'Genetics'),
 (10, 'Toxicology'),
 (11, 'Immunology'),
 (12, 'Virology'),
 (13, 'Process Development'),
 (14, 'Analytics'),
 (15, 'Validation'),
 (16, 'Engineering'),
 (17, 'Operations'),
 (18, 'Supply Chain'),
 (19, 'Regulatory'),
 (20, 'Fermentation');

	department_id	department_name
▶	1	Biology
	2	Chemistry
	3	Pharmacology
	4	R&D
	5	QA
	6	Manufacturing
	7	Clinical
	8	Microbiology
	9	Genetics
	10	Toxicology
	11	Immunology
	12	Virology
	13	Process Develop...
	14	Analytics
	15	Validation
	16	Engineering
	17	Operations
	18	Supply Chain
	19	Regulatory
	20	Fermentation

INSERT INTO location VALUES

(1, 'Lab A'),
 (2, 'Lab B'),
 (3, 'Lab C'),
 (4, 'Lab D'),
 (5, 'Lab E'),
 (6, 'Lab F'),
 (7, 'Lab G'),
 (8, 'Lab H'),

(9, 'Lab I'),
(10, 'Lab J'),
(11, 'Lab K'),
(12, 'Lab L'),
(13, 'Lab M'),
(14, 'Lab N'),
(15, 'Lab O'),
(16, 'Lab P'),
(17, 'Lab Q'),
(18, 'Lab R'),
(19, 'Lab S'),
(20, 'Lab T');

	location_id	location_name
▶	1	Lab A
	2	Lab B
	3	Lab C
	4	Lab D
	5	Lab E
	6	Lab F
	7	Lab G
	8	Lab H
	9	Lab I
	10	Lab J
	11	Lab K
	12	Lab L
	13	Lab M
	14	Lab N
	15	Lab O
	16	Lab P
	17	Lab Q
	18	Lab R
	19	Lab S
	20	Lab T

INSERT INTO experiment_type VALUES

(1, 'High-Throughput Screening (HTS)'),
(2, 'Target Identification Study'),
(3, 'Target Validation Assay'),
(4, 'PCR (Polymerase Chain Reaction)'),
(5, 'qPCR (Quantitative PCR)'),
(6, 'RNA Sequencing (RNA-seq)'),
(7, 'Gene Expression Analysis'),

(8, 'ELISA (Enzyme-Linked Immunosorbent Assay)'),
 (9, 'Western Blot Analysis'),
 (10, 'Enzyme Activity Assay'),
 (11, 'Binding Affinity Assay'),
 (12, 'Cell Viability Assay'),
 (13, 'Cytotoxicity Assay'),
 (14, 'Cell Culture Experiment'),
 (15, 'Flow Cytometry Analysis'),
 (16, 'Pharmacokinetics (PK) Study'),
 (17, 'Pharmacodynamics (PD) Study'),
 (18, 'Fermentation Run'),
 (19, 'Protein Purification (Chromatography)'),
 (20, 'Stability Study');

experiment_type_id	experiment_type_name
1	High-Throughput Screening (HTS)
2	Target Identification Study
3	Target Validation Assay
4	PCR (Polymerase Chain Reaction)
5	qPCR (Quantitative PCR)
6	RNA Sequencing (RNA-seq)
7	Gene Expression Analysis
8	ELISA (Enzyme-Linked Immunosorbent Assay)
9	Western Blot Analysis
10	Enzyme Activity Assay
11	Binding Affinity Assay
12	Cell Viability Assay
13	Cytotoxicity Assay
14	Cell Culture Experiment
15	Flow Cytometry Analysis
16	Pharmacokinetics (PK) Study
17	Pharmacodynamics (PD) Study
18	Fermentation Run
19	Protein Purification (Chromatography)
20	Stability Study

INSERT INTO sample_type VALUES

(1, 'Blood'),
 (2, 'Tissue'),
 (3, 'Serum'),
 (4, 'Plasma'),
 (5, 'DNA'),
 (6, 'RNA'),
 (7, 'Protein'),

(8, 'Cell Culture'),
 (9, 'Bacteria'),
 (10, 'Virus'),
 (11, 'Compound'),
 (12, 'Solution'),
 (13, 'Extract'),
 (14, 'Biomarker'),
 (15, 'Reagent'),
 (16, 'Antibody'),
 (17, 'Enzyme'),
 (18, 'Buffer'),
 (19, 'Medium'),
 (20, 'Control');

	sample_type_id	sample_type_name
▶	1	Blood
	2	Tissue
	3	Serum
	4	Plasma
	5	DNA
	6	RNA
	7	Protein
	8	Cell Culture
	9	Bacteria
	10	Virus
	11	Compound
	12	Solution
	13	Extract
	14	Biomarker
	15	Reagent
	16	Antibody
	17	Enzyme
	18	Buffer
	19	Medium
	20	Control

INSERT INTO maintenance_type VALUES

(1, 'Calibration'),
 (2, 'Preventive Maintenance'),
 (3, 'In-House Repair'),
 (4, 'Vendor Repair'),
 (5, 'Inspection'),
 (6, 'Equipment Qualification'),
 (7, 'Validation'),

(8, 'Decontamination'),
 (9, 'Sterilization'),
 (10, 'Troubleshooting'),
 (11, 'Installation'),
 (12, 'Relocation'),
 (13, 'Part Upgrade'),
 (14, 'Software Update'),
 (15, 'Performance Testing'),
 (16, 'Cleaning'),
 (17, 'Part Replacement'),
 (18, 'Equipment Decommissioning'),
 (19, 'Lubrication'),
 (20, 'Configuration');

	maintenance_type_id	maintenance_type_name
▶	1	Calibration
	2	Preventive Maintenance
	3	In-House Repair
	4	Vendor Repair
	5	Inspection
	6	Equipment Qualification
	7	Validation
	8	Decontamination
	9	Sterilization
	10	Troubleshooting
	11	Installation
	12	Relocation
	13	Part Upgrade
	14	Software Update
	15	Performance Testing
	16	Cleaning
	17	Part Replacement
	18	Equipment Decommissio...
	19	Lubrication
	20	Configuration

INSERT INTO experiment VALUES

(1, 4, 1, '2024-01-01', '2024-01-10', 'Completed'),
 (2, 5, 2, '2024-01-05', NULL, 'In Progress'),
 (3, 6, 3, '2024-01-07', NULL, 'Scheduled'),
 (4, 8, 4, '2024-01-12', NULL, 'In Progress'),
 (5, 10, 5, '2024-01-15', '2024-01-29', 'Completed'),
 (6, 18, 6, '2024-01-19', NULL, 'Scheduled'),
 (7, 14, 7, '2024-01-23', NULL, 'In Progress'),

(8, 8, 8, '2024-01-27', '2024-02-12', 'Completed'),
 (9, 7, 9, '2024-01-30', NULL, 'Scheduled'),
 (10, 1, 10, '2024-02-03', NULL, 'In Progress'),
 (11, 15, 11, '2024-02-05', '2024-02-18', 'Completed'),
 (12, 12, 12, '2024-02-09', NULL, 'Scheduled'),
 (13, 3, 13, '2024-02-11', NULL, 'In Progress'),
 (14, 9, 14, '2024-02-18', NULL, 'Scheduled'),
 (15, 16, 15, '2024-02-25', '2024-03-03', 'Completed'),
 (16, 18, 16, '2024-03-01', NULL, 'In Progress'),
 (17, 17, 17, '2024-03-03', '2024-03-14', 'Completed'),
 (18, 19, 18, '2024-03-16', NULL, 'Scheduled'),
 (19, 2, 19, '2024-03-19', NULL, 'In Progress'),
 (20, 18, 20, '2024-03-25', '2024-04-02', 'Completed');

	experiment_id	experiment_type_id	department_id	start_date	end_date	status
▶	1	4	1	2024-01-01	2024-01-10	Completed
	2	5	2	2024-01-05	NULL	In Progress
	3	6	3	2024-01-07	NULL	Scheduled
	4	8	4	2024-01-12	NULL	In Progress
	5	10	5	2024-01-15	2024-01-29	Completed
	6	18	6	2024-01-19	NULL	Scheduled
	7	14	7	2024-01-23	NULL	In Progress
	8	8	8	2024-01-27	2024-02-12	Completed
	9	7	9	2024-01-30	NULL	Scheduled
	10	1	10	2024-02-03	NULL	In Progress
	11	15	11	2024-02-05	2024-02-18	Completed
	12	12	12	2024-02-09	NULL	Scheduled
	13	3	13	2024-02-11	NULL	In Progress
	14	9	14	2024-02-18	NULL	Scheduled
	15	16	15	2024-02-25	2024-03-03	Completed
	16	18	16	2024-03-01	NULL	In Progress
	17	17	17	2024-03-03	2024-03-14	Completed
	18	19	18	2024-03-16	NULL	Scheduled
	19	2	19	2024-03-19	NULL	In Progress
	20	18	20	2024-03-25	2024-04-02	Completed

INSERT INTO asset VALUES

(1, 'Microscope', 'In Service', 'Current', 1, 3),
 (2, 'Centrifuge', 'In Service', 'Overdue', 2, 5),
 (3, 'PCR Machine', 'In Service', 'Current', 3, 2),
 (4, 'Incubator', 'Out of Service', 'Maintenance Needed', 4, 4),
 (5, 'Spectrometer', 'In Service', 'Current', 5, 1),
 (6, 'Fermenter', 'In Service', 'Overdue', 6, 6),
 (7, 'Freezer', 'In Service', 'Current', 7, 3),

(8, 'Flow Cytometer', 'Out of Service', 'Maintenance Needed', 8, 7),
 (9, 'Sequencer', 'In Service', 'Overdue', 9, 8),
 (10, 'Balance', 'In Service', 'Current', 10, 2),
 (11, 'Chromatography System', 'In Service', 'Current', 11, 9),
 (12, 'Autoclave', 'In Service', 'Overdue', 12, 4),
 (13, 'Bioreactor', 'Out of Service', 'Maintenance Needed', 13, 6),
 (14, 'Plate Reader', 'In Service', 'Current', 14, 5),
 (15, 'pH Meter', 'In Service', 'Current', 15, 7),
 (16, 'Fume Hood', 'In Service', 'Overdue', 16, 8),
 (17, 'Shaker', 'In Service', 'Current', 17, 9),
 (18, 'Reactor Vessel', 'Out of Service', 'Maintenance
 Needed', 18, 10),
 (19, 'Cooling System', 'In Service', 'Overdue', 19, 6),
 (20, 'Fermentation Tank', 'In Service', 'Current', 20, 6);

	asset_id	asset_type	asset_status	maintenance_status	department_id	location_id
▶	1	Microscope	In Service	Current	1	3
	2	Centrifuge	In Service	Overdue	2	5
	3	PCR Machine	In Service	Current	3	2
	4	Incubator	Out of Service	Maintenance Needed	4	4
	5	Spectrometer	In Service	Current	5	1
	6	Fermenter	In Service	Overdue	6	6
	7	Freezer	In Service	Current	7	3
	8	Flow Cytometer	Out of Service	Maintenance Needed	8	7
	9	Sequencer	In Service	Overdue	9	8
	10	Balance	In Service	Current	10	2
	11	Chromatograp...	In Service	Current	11	9
	12	Autoclave	In Service	Overdue	12	4
	13	Bioreactor	Out of Service	Maintenance Needed	13	6
	14	Plate Reader	In Service	Current	14	5
	15	pH Meter	In Service	Current	15	7
	16	Fume Hood	In Service	Overdue	16	8
	17	Shaker	In Service	Current	17	9
	18	Reactor Vessel	Out of Service	Maintenance Needed	18	10
	19	Cooling System	In Service	Overdue	19	6
	20	Fermentation T...	In Service	Current	20	6

INSERT INTO sample VALUES

(1, 3, 5, '2024-01-08'),
 (2, 1, 11, '2024-01-06'),
 (3, 5, 6, '2024-01-16'),
 (4, 2, 8, '2024-01-13'),
 (5, 8, 7, '2024-01-28'),
 (6, 3, 19, '2024-01-20'),
 (7, 10, 10, '2024-01-24'),

(8, 7, 3, '2024-01-28'),
 (9, 5, 6, '2024-01-31'),
 (10, 12, 11, '2024-02-04'),
 (11, 8, 16, '2024-02-06'),
 (12, 15, 4, '2024-02-10'),
 (13, 11, 7, '2024-02-12'),
 (14, 18, 2, '2024-02-19'),
 (15, 10, 17, '2024-02-26'),
 (16, 20, 20, '2024-03-02'),
 (17, 14, 14, '2024-03-04'),
 (18, 18, 19, '2024-03-17'),
 (19, 6, 7, '2024-03-20'),
 (20, 15, 12, '2024-03-26');

	sample_id	experiment_id	sample_type_id	collect_sample_date
▶	1	3	5	2024-01-08
	2	1	11	2024-01-06
	3	5	6	2024-01-16
	4	2	8	2024-01-13
	5	8	7	2024-01-28
	6	3	19	2024-01-20
	7	10	10	2024-01-24
	8	7	3	2024-01-28
	9	5	6	2024-01-31
	10	12	11	2024-02-04
	11	8	16	2024-02-06
	12	15	4	2024-02-10
	13	11	7	2024-02-12
	14	18	2	2024-02-19
	15	10	17	2024-02-26
	16	20	20	2024-03-02
	17	14	14	2024-03-04
	18	18	19	2024-03-17
	19	6	7	2024-03-20
	20	15	12	2024-03-26

INSERT INTO maintenance VALUES

(1, 2, 1, '2024-01-05', 'Passed'),
 (2, 4, 10, '2024-01-07', 'Repair'),
 (3, 1, 5, '2024-01-09', 'Passed'),
 (4, 6, 2, '2024-01-14', 'Failed'),
 (5, 3, 6, '2024-01-18', 'Passed'),
 (6, 8, 16, '2024-01-22', 'Repair'),
 (7, 5, 3, '2024-01-26', 'Repair'),

(8, 10, 1, '2024-01-29', 'Passed'),
 (9, 11, 15, '2024-02-01', 'Failed'),
 (10, 7, 5, '2024-02-05', 'Passed'),
 (11, 13, 17, '2024-02-08', 'Repair'),
 (12, 9, 4, '2024-02-11', 'Failed'),
 (13, 15, 2, '2024-02-13', 'Passed'),
 (14, 12, 8, '2024-02-20', 'Passed'),
 (15, 17, 10, '2024-02-27', 'Repair'),
 (16, 6, 19, '2024-03-03', 'Passed'),
 (17, 18, 11, '2024-03-05', 'Repair'),
 (18, 14, 14, '2024-03-18', 'Passed'),
 (19, 19, 7, '2024-03-21', 'Failed'),
 (20, 2, 20, '2024-03-27', 'Passed');

	maintenance_id	asset_id	maintenance_type_id	maintenance_date	result
▶	1	2	1	2024-01-05	Passed
	2	4	10	2024-01-07	Repair
	3	1	5	2024-01-09	Passed
	4	6	2	2024-01-14	Failed
	5	3	6	2024-01-18	Passed
	6	8	16	2024-01-22	Repair
	7	5	3	2024-01-26	Repair
	8	10	1	2024-01-29	Passed
	9	11	15	2024-02-01	Failed
	10	7	5	2024-02-05	Passed
	11	13	17	2024-02-08	Repair
	12	9	4	2024-02-11	Failed
	13	15	2	2024-02-13	Passed
	14	12	8	2024-02-20	Passed
	15	17	10	2024-02-27	Repair
	16	6	19	2024-03-03	Passed
	17	18	11	2024-03-05	Repair
	18	14	14	2024-03-18	Passed
	19	19	7	2024-03-21	Failed
	20	2	20	2024-03-27	Passed

INSERT INTO asset_experiment VALUES

(1, 3, '2024-01-01', '2024-01-10'),
 (1, 1, '2024-01-01', '2024-01-10'),
 (2, 5, '2024-01-05', NULL),
 (3, 2, '2024-01-07', NULL),
 (3, 6, '2024-01-07', NULL),
 (4, 4, '2024-01-12', NULL),
 (5, 8, '2024-01-15', '2024-01-29'),
 (5, 2, '2024-01-16', '2024-01-28'),

(6, 6, '2024-01-19', NULL),
 (7, 7, '2024-01-23', NULL),
 (7, 3, '2024-01-23', NULL),
 (8, 8, '2024-01-27', '2024-02-12'),
 (9, 9, '2024-01-30', NULL),
 (10, 10, '2024-02-03', NULL),
 (11, 11, '2024-02-05', '2024-02-18'),
 (11, 1, '2024-02-05', '2024-02-18'),
 (12, 12, '2024-02-09', NULL),
 (13, 13, '2024-02-11', NULL),
 (14, 14, '2024-02-18', NULL),
 (14, 5, '2024-02-18', NULL),
 (15, 15, '2024-02-25', '2024-03-03'),
 (16, 7, '2024-03-01', NULL),
 (17, 17, '2024-03-03', '2024-03-14'),
 (17, 10, '2024-03-03', '2024-03-14'),
 (18, 18, '2024-03-16', NULL),
 (19, 4, '2024-03-19', NULL),
 (20, 20, '2024-03-25', '2024-04-02');

	experiment_id	asset_id	usage_start_date	usage_end_date
▶	1	1	2024-01-01	2024-01-10
	1	3	2024-01-01	2024-01-10
	2	5	2024-01-05	NULL
	3	2	2024-01-07	NULL
	3	6	2024-01-07	NULL
	4	4	2024-01-12	NULL
	5	2	2024-01-16	2024-01-28
	5	8	2024-01-15	2024-01-29
	6	6	2024-01-19	NULL
	7	3	2024-01-23	NULL
	7	7	2024-01-23	NULL
	8	8	2024-01-27	2024-02-12
	9	9	2024-01-30	NULL
	10	10	2024-02-03	NULL
	11	1	2024-02-05	2024-02-18
	11	11	2024-02-05	2024-02-18
	12	12	2024-02-09	NULL
	13	13	2024-02-11	NULL
	14	5	2024-02-18	NULL
	14	14	2024-02-18	NULL
	15	15	2024-02-25	2024-03-03
	16	7	2024-03-01	NULL
	17	10	2024-03-03	2024-03-14
	17	17	2024-03-03	2024-03-14
	18	18	2024-03-16	NULL
	19	4	2024-03-19	NULL
	20	20	2024-03-25	2024-04-02

STEP 3 | Write at least 5 - 8 SQL queries on your database to show that your database is working perfectly.

-- Simple (find assets that are out of service)

SELECT *

FROM asset

WHERE asset_status = 'Out of Service';

	asset_id	asset_type	asset_status	maintenance_status	department_id	location_id
▶	4	Incubator	Out of Service	Maintenance Needed	4	4
	8	Flow Cytometer	Out of Service	Maintenance Needed	8	7
	13	Bioreactor	Out of Service	Maintenance Needed	13	6
	18	Reactor Vessel	Out of Service	Maintenance Needed	18	10

-- Aggregate (count the number of experiments based on status)

SELECT status, COUNT(*) AS total_experiments

FROM experiment

GROUP BY status;

	status	total_experiments
▶	Completed	7
	In Progress	7
	Scheduled	6

-- INNER JOIN (what sample type is each sample)

SELECT s.sample_id, s.experiment_id, st.sample_type_name

FROM sample AS s

JOIN sample_type AS st

ON s.sample_type_id = st.sample_type_id;

	sample_id	experiment_id	sample_type_name
▶	1	3	DNA
	2	1	Compound
	3	5	RNA
	4	2	Cell Culture
	5	8	Protein
	6	3	Medium
	7	10	Virus
	8	7	Serum
	9	5	RNA
	10	12	Compound
	11	8	Antibody
	12	15	Plasma
	13	11	Protein
	14	18	Tissue
	15	10	Enzyme
	16	20	Control
	17	14	Biomarker
	18	18	Medium
	19	6	Protein
	20	15	Solution

-- Outer Join (experiments that have and don't have sample records)

SELECT e.experiment_id, e.status, s.sample_id

FROM experiment AS e

LEFT JOIN sample AS s

ON e.experiment_id = s.experiment_id;

	experiment_id	status	sample_id
▶	1	Completed	2
	2	In Progress	4
	3	Scheduled	1
	3	Scheduled	6
	4	In Progress	NULL
	5	Completed	3
	5	Completed	9
	6	Scheduled	19
	7	In Progress	8
	8	Completed	5
	8	Completed	11
	9	Scheduled	NULL
	10	In Progress	7
	10	In Progress	15
	11	Completed	13
	12	Scheduled	10
	13	In Progress	NULL
	14	Scheduled	17
	15	Completed	12
	15	Completed	20
	16	In Progress	NULL
	17	Completed	NULL
	18	Scheduled	14
	18	Scheduled	18
	19	In Progress	NULL
	20	Completed	16

-- Nested query (assets with overdue maintenance that experiments used)

```
SELECT DISTINCT e.experiment_id
FROM experiment AS e
WHERE e.experiment_id IN (
    SELECT ae.experiment_id
    FROM asset_experiment ae
    JOIN asset AS a
    ON ae.asset_id = a.asset_id
    WHERE a.maintenance_status = 'Overdue');
```

	experiment_id
▶	3
	5
	6
	9
	12

-- Correlated query (find experiments that took longer than the average completed experiment)

```
SELECT exp1.experiment_id,
    exp1.start_date,
    exp1.end_date,
    DATEDIFF(exp1.end_date, exp1.start_date) AS total_days
```

```

FROM experiment AS exp1

WHERE exp1.end_date IS NOT NULL

AND DATEDIFF(exp1.end_date, exp1.start_date) > (SELECT
AVG(DATEDIFF(exp2.end_date, exp2.start_date))

FROM experiment AS exp2

WHERE exp2.end_date IS NOT NULL);

```

	experiment_id	start_date	end_date	total_days
▶	5	2024-01-15	2024-01-29	14
	8	2024-01-27	2024-02-12	16
	11	2024-02-05	2024-02-18	13

```

-- >=ALL/>ANY/Exists/Not Exists (experiments that have no sample records)

```

```

SELECT *

```

```

FROM experiment AS e

```

```

WHERE NOT EXISTS (

```

```

    SELECT 1

```

```

    FROM sample AS s

```

```

    WHERE s.experiment_id = e.experiment_id);

```

▶	1a	5	1a	505+03-1a	NOT	In Progress
	1a	1a	1a	505+03-03	505+03-1a	Completed
	1e	1a	1e	505+03-01	NOT	In Progress
	1a	3	1a	505+05-11	NOT	In Progress
	a	1	a	505+01-30	NOT	Completed
▶	a	8	a	505+01-15	NOT	In Progress
	experiment_id	experiment_id	experiment_id	start_date	end_date	status

```

-- UNION (assets that are overdue or maintenance needed)

```

```

SELECT asset_id, maintenance_status

```

```

FROM asset

```

```

WHERE maintenance_status = 'Overdue'

```

```

UNION

```

```

SELECT asset_id, maintenance_status

```

```

FROM asset

```

```

WHERE maintenance_status = 'Maintenance Needed';

```

	asset_id	maintenance_status
▶	2	Overdue
	6	Overdue
	9	Overdue
	12	Overdue
	16	Overdue
	19	Overdue
	4	Maintenance Needed
	8	Maintenance Needed
	13	Maintenance Needed
	18	Maintenance Needed

-- Subquery in FROM (experiments that have the most samples)

```
SELECT e.experiment_id,
       e.department_id,
       sc.sample_count
```

```
FROM experiment AS e
```

```
JOIN (
```

```
    SELECT experiment_id, COUNT(*) AS
    sample_count
```

```
    FROM sample
```

```
    GROUP BY experiment_id) AS sc
```

```
ON e.experiment_id = sc.experiment_id
```

```
ORDER BY sc.sample_count DESC;
```

	experiment_id	department_id	sample_count
▶	3	3	2
	5	5	2
	8	8	2
	10	10	2
	15	15	2
	18	18	2
	1	1	1
	2	2	1
	6	6	1
	7	7	1
	11	11	1
	12	12	1
	14	14	1
	20	20	1

-- Subquery in SELECT (find experiments with no samples)

```
SELECT e.experiment_id,
```

```
       e.department_id,
```

```
       (SELECT COUNT(*)
```

```
        FROM sample AS s
```

```
        WHERE s.experiment_id = e.experiment_id) AS sample_count
```

```
FROM experiment AS e
```

```
WHERE (
```

```
SELECT COUNT(*)  
FROM sample AS s  
WHERE s.experiment_id = e.experiment_id) = 0;
```

	experiment_id	department_id	sample_count
►	4	4	0
	9	9	0
	13	13	0
	16	16	0
	17	17	0
	19	19	0

STEP 4| Create an application using a Jupyter notebook (.ipynb) or R Markdown (.Rmd).

```
[1]: !pip install mysql-connector-python
```

Requirement already satisfied: mysql-connector-python in c:\users\charl\anaconda3\lib\site-packages (9.6.0)

```
[2]: import mysql.connector
import pandas as pd
import matplotlib.pyplot as plt
```

```
[3]: conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="Hithere99!",
    database="milestone2db")
```

```
[4]: import warnings
warnings.filterwarnings("ignore")

#STEP 4| Create an application using a Jupyter notebook (.ipynb) or R Markdown (.Rmd).

#Aggregate query
#count the number of experiments based on status

query_status = """
SELECT status, COUNT(*) AS total_experiments
FROM experiment
GROUP BY status;
"""

df_status = pd.read_sql(query_status, conn)
df_status
```

```
[4]:
```

	status	total_experiments
0	Completed	7
1	In Progress	7
2	Scheduled	6

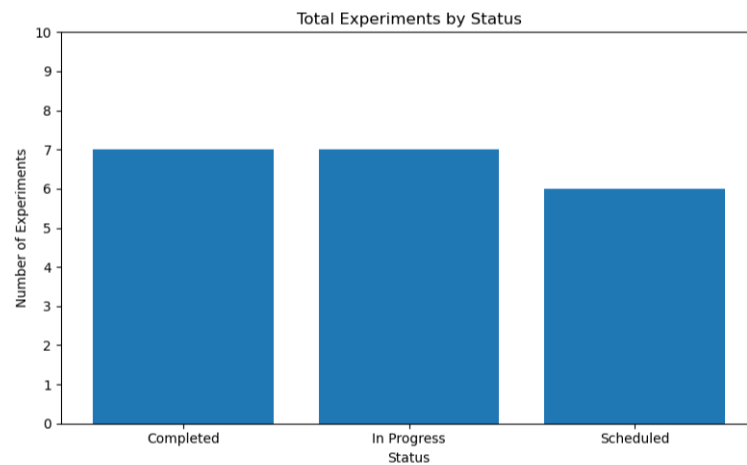
```
[5]: #Bar Chart: count the number of experiments based on status
```

```
plt.figure(figsize=(8,5))
plt.bar(df_status["status"], df_status["total_experiments"])

plt.title("Total Experiments by Status")
plt.xlabel("Status")
plt.ylabel("Number of Experiments")

plt.ylim(0, 10)
plt.yticks(range(0, 11, 1))

plt.tight_layout()
plt.show()
```




```
[6]: #Subquery in FROM
#experiments that have the most samples

query_samples = """
SELECT e.experiment_id,
       e.department_id,
       sc.sample_count
FROM experiment AS e
JOIN (
    SELECT experiment_id, COUNT(*) AS sample_count
    FROM sample
    GROUP BY experiment_id) AS sc
ON e.experiment_id = sc.experiment_id
ORDER BY sc.sample_count DESC;
"""

df_samples = pd.read_sql(query_samples, conn)
df_samples
```

```
[6]:
```

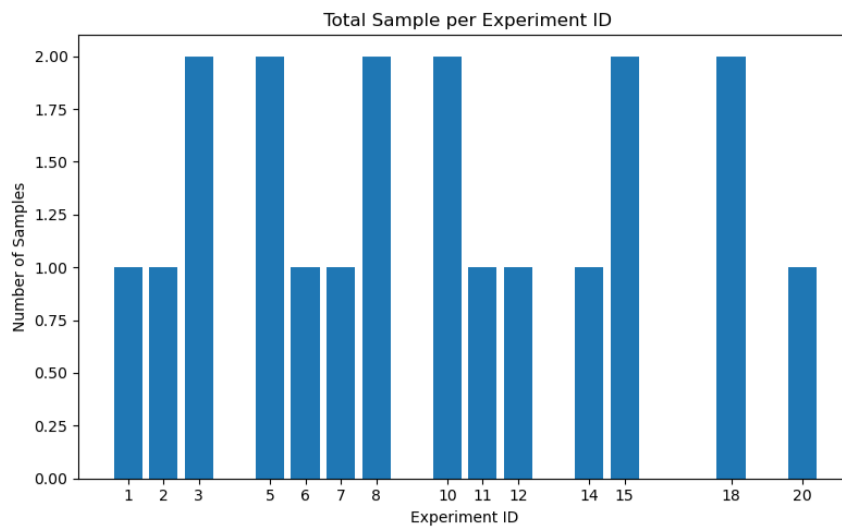
	experiment_id	department_id	sample_count
0	3	3	2
1	5	5	2
2	8	8	2
3	10	10	2
4	15	15	2
5	18	18	2
6	1	1	1
7	2	2	1
8	6	6	1
9	7	7	1
10	11	11	1
11	12	12	1
12	14	14	1
13	20	20	1

```
[7]: #Bar Chart:experiments that have the most samples

plt.figure(figsize=(8,5))
plt.bar(df_samples["experiment_id"], df_samples["sample_count"])

plt.title(" Total Sample per Experiment ID")
plt.xlabel("Experiment ID")
plt.ylabel("Number of Samples")

plt.xticks(df_samples["experiment_id"])
plt.tight_layout()
plt.show()
```



```
[8]: # Correlated query
#experiments that took longer than the average completed experiment time(days)

query_time = """
SELECT exp1.experiment_id,
       exp1.start_date,
       exp1.end_date,
       DATEDIFF(exp1.end_date, exp1.start_date) AS total_days
FROM experiment AS exp1
WHERE exp1.end_date IS NOT NULL
      AND DATEDIFF(exp1.end_date, exp1.start_date) > (SELECT AVG(DATEDIFF(exp2.end_date, exp2.start_date))
FROM experiment AS exp2
WHERE exp2.end_date IS NOT NULL);
"""

df_time = pd.read_sql(query_time, conn)
df_time
```

```
[8]:
```

	experiment_id	start_date	end_date	total_days
0	5	2024-01-15	2024-01-29	14
1	8	2024-01-27	2024-02-12	16
2	11	2024-02-05	2024-02-18	13

```
[9]: # Bar Chart:experiments that took longer than the average completed experiment time(days)

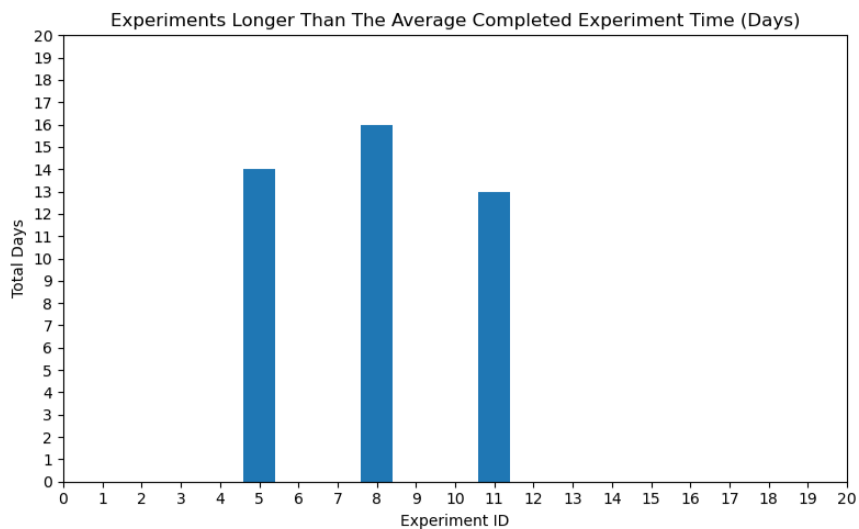
plt.figure(figsize=(8,5))
plt.bar(df_time["experiment_id"], df_time["total_days"])

plt.title("Experiments Longer Than The Average Completed Experiment Time (Days)")
plt.xlabel("Experiment ID")
plt.ylabel("Total Days")

plt.xlim(0, 20)
plt.ylim(0, 20)

plt.xticks(range(0, 21, 1))
plt.yticks(range(0, 21, 1))

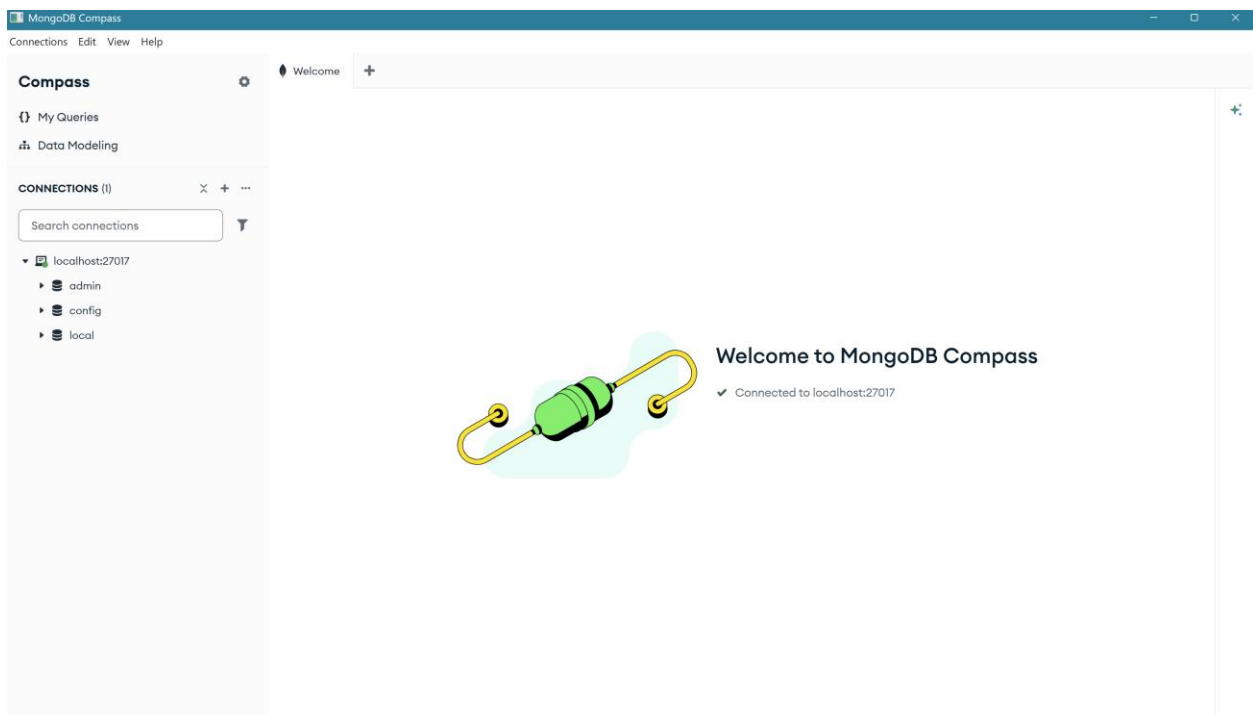
plt.tight_layout()
plt.show()
```



1. Environment Setup

For milestone 3, I used MongoDB Compass as my NoSQL. I chose MongoDB because it allows data to be stored in collections of documents rather than relational tables, while supporting filtering, projection, sorting, and aggregation queries. I installed MongoDB Compass, connected to my local MongoDB server, and successfully connected.

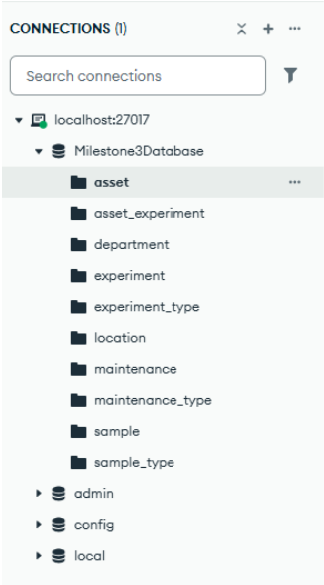
Screenshot (connected to MongoDB Compass):



2. Database Setup

I created milestone 3 database in MongoDB using the same entities and reference data from my relational database in milestone 2. In Milestone 2 my reference tables were department, location, experiment type, sample type, and maintenance type, along with the main tables experiment, asset, sample, maintenance, and asset_experiment which I used as the collections in MongoDB.

Screenshot (database setup in MongoDB):



3. Data Import

The dataset that I imported was from milestone 2 which is my original data when creating the relational database in MySQL which includes tables such as department, location, experiment_type, sample_type, maintenance_type, experiment, asset, sample, maintenance, and asset_experiment. I converted the relational tables into CSV files and imported each file into the corresponding MongoDB collection. After importing, I verified that the data was successfully loaded by viewing the documents within each collection.

Screenshot (milestone3database collections):

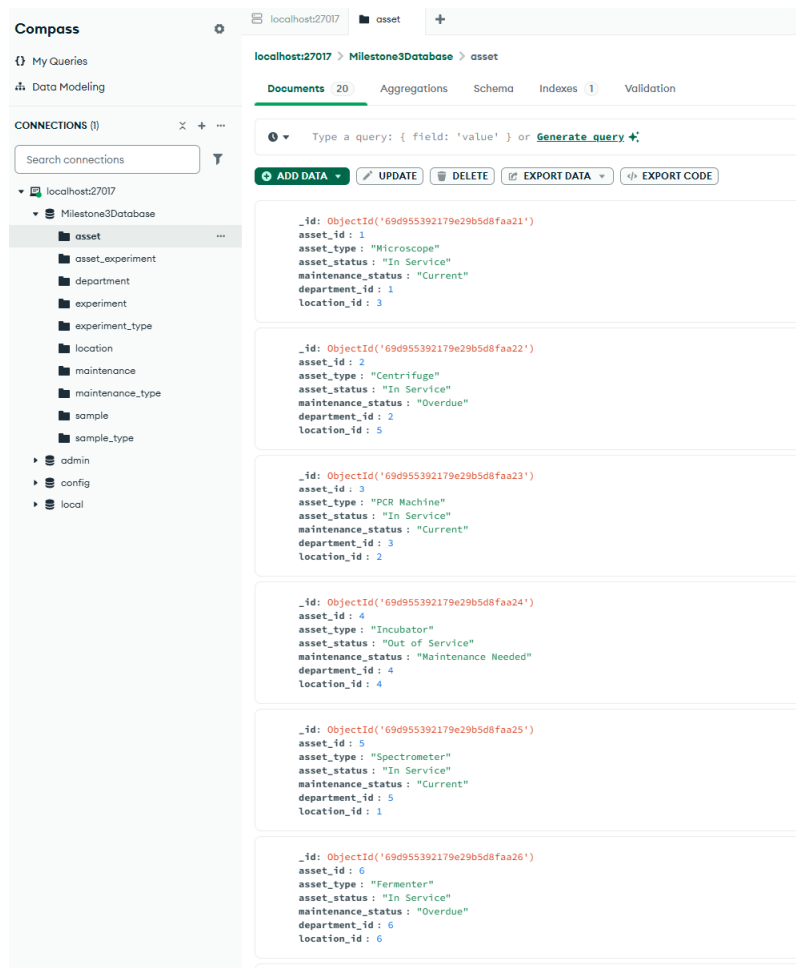
Compass

localhost:27017 Milestone3Database

Open MongoDB shell Create collection Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
asset	-	4.30 kB	3.28 kB	20	163.00 B	1	4.10 kB
asset_experiment	-	4.30 kB	2.46 kB	27	90.00 B	1	4.10 kB
department	-	4.30 kB	1.44 kB	20	72.00 B	1	4.10 kB
experiment	-	4.30 kB	2.66 kB	20	133.00 B	1	4.10 kB
experiment_type	-	4.30 kB	1.96 kB	20	97.00 B	1	4.10 kB
location	-	4.30 kB	1.28 kB	20	64.00 B	1	4.10 kB
maintenance	-	4.30 kB	2.62 kB	20	126.00 B	1	4.10 kB
maintenance_type	-	4.30 kB	1.79 kB	20	89.00 B	1	4.10 kB
sample	-	4.30 kB	2.10 kB	20	105.00 B	1	4.10 kB
sample_type	-	4.30 kB	1.43 kB	20	71.00 B	1	4.10 kB

Screenshot (example of populated data for collection asset from milestone3database):



4. Queries (Minimum 3)

1. return assets that the maintenance status is overdue

Filters the asset collection based on the maintenance_status “Overdue” then sorts the outcome by asset_id

Query:

```
db.asset.find(
{ maintenance_status: "Overdue" },
{ _id: 0, asset_id: 1, asset_type: 1, maintenance_status: 1, department_id: 1, location_id: 1 }
).sort({ asset_id: 1 })
```

localhost:27017 asset +

localhost:27017 > Milestone3Database > asset

Documents (20) Aggregations Schema Indexes (1) Validation

```

{ maintenance_status: "Overdue" }

Project { _id: 0, asset_id: 1, asset_type: 1, maintenance_status: 1, department_id: 1, location_id: 1 }

Sort { asset_id: 1 }

Collation { locale: 'simple' }

Index Hint { field: -1 } or "indexName"

```

Output:

```

asset_id : 2
asset_type : "Centrifuge"
maintenance_status : "Overdue"
department_id : 2
location_id : 5

```

```

asset_id : 6
asset_type : "Fermenter"
maintenance_status : "Overdue"
department_id : 6
location_id : 6

```

```

asset_id : 9
asset_type : "Sequencer"
maintenance_status : "Overdue"
department_id : 9
location_id : 8

```

```

asset_id : 12
asset_type : "Autoclave"
maintenance_status : "Overdue"
department_id : 12
location_id : 4

```

```

asset_id : 16
asset_type : "Fume Hood"
maintenance_status : "Overdue"
department_id : 16
location_id : 8

```

```

asset_id : 19
asset_type : "Cooling System"
maintenance_status : "Overdue"
department_id : 19
location_id : 6

```

2. Return the most popular sample types

Joins the sample collection with the sample_type collection using \$lookup followed by groups and counts each sample_type then sorted in descending order (lets us see which sample type is the most popular and going down in popularity)

Query:

```

db.sample.aggregate([
{
  $lookup: {

```

```

from: "sample_type",

localField: "sample_type_id",

foreignField: "sample_type_id",

as: "sample_type_info" }},

{ $unwind: "$sample_type_info" },

{

  $group: {

    _id: "$sample_type_info.sample_type_name",

    total_count: { $sum: 1 } }},

{ $sort: { total_count: -1 } })

```

Stage 1: \$lookup

```

1 //
2 // from: The target collection.
3 // localField: The local join field.
4 // foreignField: The target join field.
5 // as: The name for the results.
6 // pipeline: Optional pipeline to run on
7 // (Optional) variables to use in the
8 //
9 {
10   from: "sample_type",
11   localField: "sample_type_id",
12   foreignField: "sample_type_id",
13   as: "sample_type_info"
14 }

```

Output preview after \$lookup if stage (Sample of 10 documents)

<pre> _id: ObjectId("60d955472179629b5d8faa36") sample_id: 1 experiment_id: 3 sample_type_id: 5 collect_sample_date: 2024-01-06T00:00:00.000Z sample_type_info: Array (1) </pre>	<pre> _id: ObjectId("60d955472179629b5d8faa37") sample_id: 2 experiment_id: 1 sample_type_id: 11 collect_sample_date: 2024-01-06T00:00:00.000Z sample_type_info: Array (1) </pre>	<pre> _id: ObjectId("60d955472179629b5d8faa38") sample_id: 3 experiment_id: 5 sample_type_id: 0 collect_sample_date: 2024-01-16T00:00:00.000Z sample_type_info: Array (1) </pre>	<pre> _id: ObjectId("60d955472179629b5d8faa39") sample_id: 4 experiment_id: 2 sample_type_id: 9 collect_sample_date: 2024-01-13T00:00:00.000Z sample_type_info: Array (1) </pre>	<pre> _id: ObjectId("60d955472179629b5d8faa3a") sample_id: 5 experiment_id: 8 sample_type_id: 1 collect_sample_date: 2024-01-13T00:00:00.000Z sample_type_info: Array (1) </pre>
--	---	--	--	--

Stage 2: \$unwind

```

1 //
2 // path: Path to the array field.
3 // includeEmptyArrays: Optional name for
4 // preserveNullAndEmptyArrays: Optional
5 // toggle to unwind null and empty values
6 //
7 "$sample_type_info"

```

Output preview after \$unwind if stage (Sample of 10 documents)

<pre> _id: ObjectId("60d955472179629b5d8faa36") sample_id: 1 experiment_id: 3 sample_type_id: 5 collect_sample_date: 2024-01-06T00:00:00.000Z sample_type_info: Object </pre>	<pre> _id: ObjectId("60d955472179629b5d8faa37") sample_id: 2 experiment_id: 1 sample_type_id: 11 collect_sample_date: 2024-01-06T00:00:00.000Z sample_type_info: Object </pre>	<pre> _id: ObjectId("60d955472179629b5d8faa38") sample_id: 3 experiment_id: 5 sample_type_id: 0 collect_sample_date: 2024-01-16T00:00:00.000Z sample_type_info: Object </pre>	<pre> _id: ObjectId("60d955472179629b5d8faa39") sample_id: 4 experiment_id: 2 sample_type_id: 9 collect_sample_date: 2024-01-13T00:00:00.000Z sample_type_info: Object </pre>	<pre> _id: ObjectId("60d955472179629b5d8faa3a") sample_id: 5 experiment_id: 8 sample_type_id: 1 collect_sample_date: 2024-01-13T00:00:00.000Z sample_type_info: Object </pre>
---	--	---	---	---

Stage 3: \$group

```

1 //
2 // _id: The id of the group.
3 // fields: The first field name.
4 //
5 {
6   _id: "$sample_type_info.sample_type_name",
7   total_count: { $sum: 1 }
8 }

```

Output preview after \$group if stage (Sample of 10 documents)

<pre> _id: "DNA" total_count: 1 </pre>	<pre> _id: "Compound" total_count: 2 </pre>	<pre> _id: "Protein" total_count: 3 </pre>	<pre> _id: "Enzyme" total_count: 1 </pre>	<pre> _id: "Ant" total_count: 1 </pre>
--	---	--	---	--

Stage 4: \$sort

```

1 //
2 // Provide any number of field/order pair
3 //
4 { total_count: -1 }

```

Output preview after \$sort if stage (Sample of 10 documents)

<pre> _id: "Protein" total_count: 3 </pre>	<pre> _id: "Medium" total_count: 2 </pre>	<pre> _id: "Compound" total_count: 2 </pre>	<pre> _id: "DNA" total_count: 2 </pre>	<pre> _id: "RNA" total_count: 1 </pre>
--	---	---	--	--

Output:

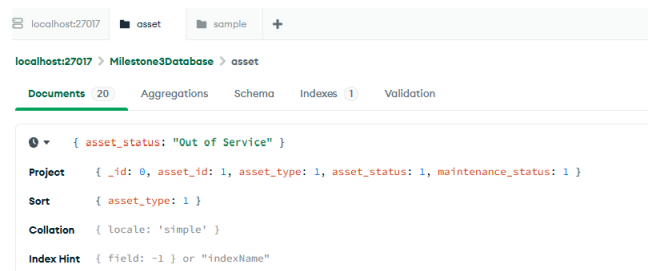


3. return assets that are out of service

Filters the asset collection based on `asset_status` “Out of Service” then sorts by `asset_type`

Query:

```
db.asset.find(  
  { asset_status: "Out of Service" },  
  { _id: 0, asset_id: 1, asset_type: 1, asset_status: 1, maintenance_status: 1 }  
) .sort({ asset_type: 1 })
```



Output:

